

FastAPI Backend and End to End System Integration

1. Introduction

Phishing email detection has progressed from manually maintained rules toward computational pipelines that combine natural language processing, supervised classification and human facing interpretation. However, a high performing classifier alone does not constitute a usable defensive system. Email data must be acquired, validated and transformed consistently; inference must be made accessible to distinct interfaces; and results must be communicated in a form that users can understand without misinterpreting confidence or explanations. Backend application programming interfaces (APIs) therefore represent a consequential, although sometimes under examined, part of operational machine learning research.

This review examines FastAPI and end to end integration for a prototype that couples a DeBERTa-v3-large gated-hybrid email classifier with SHAP based explanations, a Streamlit interface and a Chrome browser extension. It evaluates API oriented deployment patterns against alternative architectures, critically considers the reliability and security of integrated inference services, and derives defensible design and evaluation requirements. The review treats framework documentation and security standards as practitioner evidence rather than presenting them as peer reviewed empirical proof.

2. Background and key concepts

An inference serving backend separates model computation from user interface concerns. A client sends structured email data typically sender, subject and body to a server, which applies preprocessing, constructs engineered features, executes the classifier and returns a prediction. Such separation is especially relevant when the same model is consumed by heterogeneous clients: a Python based dashboard and a JavaScript browser extension can use a common service rather than independently implementing preprocessing and model loading. This is an architectural advantage, not a guarantee that the clients produce identical results; equivalence requires a shared input contract and explicit testing.

FastAPI is a Python web framework that uses type annotations and Pydantic compatible schemas to support request validation and OpenAPI documentation (FastAPI n.d. a). Typed contracts are valuable for rejecting malformed payloads, specifying required versus optional fields, and making errors observable. The HTTP interface can expose distinct responsibilities for example, /predict for classification, /explain for explanation generation and /health for service readiness while maintaining a single canonical preprocessing and model inference implementation. These endpoint names reflect the project architecture described by its development materials; their exact behaviour must be checked against the deployed code.

Asynchronous web programming requires careful interpretation. FastAPI supports asynchronous endpoints and conventional synchronous functions (FastAPI n.d. b), but an async route does not automatically accelerate CPU or GPU intensive transformer inference. Concurrency and throughput depend on model execution, worker configuration, batching, hardware memory and contention. SHAP explanations can impose additional computational cost because an explainer may evaluate numerous perturbations or background samples. Consequently, prediction and explanation may warrant separate service paths, time budgets or resource limits.

End to end integration also introduces data governance questions. Email bodies can contain private correspondence, personal identifiers, embedded links and potentially malicious text. The server must treat client provided text as untrusted data, avoid unnecessary retention, and ensure that explanations do not inadvertently reproduce sensitive content. Browser to API communication additionally depends on permitted origins, network configuration and clear handling of request failures.

3. Review of existing research

Foundational software architecture research identifies modularity and separation of concerns as means of managing system complexity, while service oriented and microservice literature considers how independently addressable components can be composed. Dragoni et al. (2017) describe microservices as an approach to decomposing software into independently deployable services, but note the operational complexity introduced by distributed communication and service management. For a small research prototype, a single inference API serving multiple interfaces may capture useful separation without the overhead of a full microservice ecosystem.

Sculley et al. (2015) argue that machine learning systems accrue technical debt beyond the predictive model, including hidden feedback loops, data dependencies, configuration complexity and glue code. This is directly applicable to phishing detection: the deployed system can fail even if offline evaluation is strong, for example when browser extraction changes field boundaries or when preprocessing differs between training and inference. Their analysis motivates versioned model artefacts, reproducible feature construction, dependency management and integration tests. However, the paper is conceptual and does not establish the performance of any particular FastAPI implementation.

Breck et al. (2017) propose a rubric for testing and monitoring production ML systems, covering data, models, infrastructure and operational practices. Their contribution shifts evaluation beyond aggregate classification metrics toward testable system properties. Applied to the present project, this suggests validating schema constraints, missing field behaviour, repeatability, consistent outputs across clients and error reporting. The rubric does not prescribe a universal deployment stack and should be adapted to the limited scale of a university prototype.

Explainability literature further complicates backend design. Lundberg and Lee (2017) formalise SHAP as a local additive attribution framework, but the computational pathway used to obtain an explanation can differ substantially from ordinary classification. A backend must preserve the exact class, output scale, feature representation and explainer assumptions to prevent explanations from being presented as direct probabilities or causal reasons. The existence of an /explain endpoint alone is insufficient evidence of explanation faithfulness or stable attribution.

API security literature supplies another essential perspective. The OWASP API Security Top 10 (OWASP 2023) highlights risks including broken authentication, misconfiguration and unrestricted resource consumption. Resource exhaustion is especially relevant to transformer inference and SHAP explanation endpoints, where repeated or oversized requests may consume substantial memory and compute. These standards identify plausible threat classes rather than demonstrate that the project is vulnerable; an implementation specific security review is required.

Finally, NIST's AI Risk Management Framework (NIST 2023) emphasises validity, reliability, security, transparency and ongoing monitoring as characteristics of trustworthy AI. Although the framework is not a phishing specific API study, it offers a useful evaluative lens: successful request

handling should not be confused with calibrated predictions, reliable explanations or safe user decisions. This broader perspective prevents end to end demonstrations from being treated as proof of real world effectiveness.

4. Comparison of existing methods or solutions

Architectural options differ in development overhead, cross client reuse, operational flexibility and security exposure. A monolithic dashboard may be easiest to prototype, but directly embedding inference in the interface couples model dependencies to presentation logic. A separate REST API permits multiple clients to share the same classifier, while a managed cloud inference service can externalise scaling at the cost of vendor dependence and additional privacy considerations. A fully decomposed microservice system offers independent scaling of components but is difficult to justify without demonstrated demand or organizational requirements.

Approach	Strengths	Limitations	Suitability
UI embedded inference	Fast local prototyping; fewer network boundaries	Duplicate logic across interfaces; tighter coupling	Useful baseline, limited multi-client reuse
FastAPI REST backend	Shared contracts; Python ML integration; OpenAPI support	Network failure modes; service hardening required	Strong fit for this prototype
Managed inference platform	Potential autoscaling and monitoring	Cost, privacy, provider dependence	Future deployment option
Multiple microservices	Independent scaling and ownership	Operational and orchestration overhead	Premature without load evidence

5. Research gaps and limitations

A recurring gap between machine learning papers and working systems is the limited reporting of deployment assumptions. Studies often prioritise precision, recall and F1 score while omitting input validation failures, API latency distributions, memory usage, model loading time, and explanation generation overhead. Without these measurements, reported model quality cannot be translated into interactive responsiveness or operational capacity. Moreover, latency measured on one machine cannot be generalised to different hardware or concurrent workloads.

A second gap concerns cross interface consistency. A Streamlit form and browser content script may capture different portions of an email, represent missing senders differently, or include HTML artefacts. Unless the backend canonicalises inputs and test cases compare equivalent payloads, discrepancies may be incorrectly attributed to model instability. Browser extension success in a controlled test environment also cannot establish robustness across changing commercial webmail DOM structures.

Third, explanations introduce interpretive and security risks. SHAP attributions depend on the explainer configuration, background data, feature dependence assumptions and model output

being explained. If the explanation pipeline uses a proxy model, different preprocessing or a subset of the twelve engineered features, the resulting display must be labelled accordingly. Neither a convincing visualisation nor a correct classification verifies explanation fidelity.

Fourth, prototype APIs frequently lack the hardening expected of public services: authentication where needed, request size restrictions, rate limiting, narrow CORS policies, secrets management, logging safeguards, dependency updates and transport encryption. OWASP (2023) makes clear why unrestricted resource consumption and security misconfiguration merit attention, but a review must not infer that these safeguards are absent or present without inspecting implementation and deployment settings.

Finally, usability evidence remains distinct from integration evidence. A successful screen recording shows that a particular workflow can execute; it does not quantify whether non expert users understand uncertainty, distinguish feature attribution from causation, or respond appropriately to false positives. Controlled user studies and task based evaluation would be necessary to support those stronger claims.

6. Implications for the proposed project

The reviewed literature supports a single, well defined FastAPI inference boundary serving both Streamlit and the Chrome extension. The backend should own the canonical transformation of subject and body into model text, construction of the twelve engineered features, model loading and the production of class scores. This reduces duplication and provides a clear place to enforce input limits and schema validation. Whether the present implementation fully centralises these operations should be verified against `api.py`, `predict.py` and the client code.

For classification, the `/predict` contract should define the expected fields, supported missing values, maximum lengths, output class ordering and the meaning of confidence. For explanation, `/explain` should document the explained class, output scale, feature names, baseline and SHAP settings. In the interface, “supporting” and “opposing” factors should be described relative to the output being explained, while “strongest” should refer to absolute attribution magnitude if that is how the ranking is computed. This specification guards against misleading presentation when Safe and Phishing results are compared.

An evidence based validation plan should include: identical email payloads sent from Streamlit and the extension; correct Safe and Phishing classifications on labelled examples; rejection of invalid or oversized payloads; stable and interpretable error responses; unavailable backend behavior; timing of `/predict` versus `/explain`; repeatability of outputs; and inspection of privacy sensitive logging. Cross origin settings and extension host permissions should be documented rather than assumed secure.

Performance claims should be tied to measured conditions. Report hardware, model checkpoint, input lengths, request counts, concurrency, warm versus cold starts and latency summaries (such as median and 95th percentile). For SHAP, report explanation latency separately and state the number of evaluations or background samples where applicable. These measures would permit a more credible assessment of interactive suitability than a demonstration alone.

At the present stage, the defensible contribution is the architectural integration of a gated hybrid classifier and explainability pipeline into two user facing interfaces through an API service. Claims of scalability, security compliance, general Gmail compatibility or production readiness should

remain future work statements unless directly supported by systematic testing. This distinction strengthens the academic integrity of the final report.

7. Conclusion

The literature indicates that deploying explainable phishing detection requires coordinated attention to software architecture, data contracts, model reproducibility, API security and human interpretation. FastAPI provides a pragmatic Python native mechanism for exposing inference and explanation to multiple interfaces, but its convenience does not eliminate the challenges of computationally expensive transformer models, SHAP overhead or sensitive email data.

For the proposed project, a shared backend is justified primarily by consistency and separation of responsibilities rather than unverified claims of superior speed. The strongest next step is rigorous end to end validation of equivalent inputs, error handling, explanation semantics, latency and privacy controls. Framed this way, the prototype contributes an integrated and testable research artefact while acknowledging the substantial work required before real world deployment.

References

Breck, E, Cai, S, Nielsen, E, Salib, M & Sculley, D 2017, 'The ML test score: a rubric for ML production readiness and technical debt reduction', 2017 IEEE International Conference on Big Data, IEEE, pp. 1123–1132, <https://doi.org/10.1109/BigData.2017.8258038>

Dragoni, N, Giallorenzo, S, Lafuente, AL, Mazzara, M, Montesi, F, Mustafin, R & Safina, L 2017, 'Microservices: yesterday, today, and tomorrow', in M Mazzara & B Meyer (eds), Present and ulterior software engineering, Springer, pp. 195–216, https://doi.org/10.1007/978_3_319_67425_4_12

FastAPI n.d. a, FastAPI documentation, <https://fastapi.tiangolo.com/>

FastAPI n.d. b, Concurrency and async / await, <https://fastapi.tiangolo.com/async/>

Lundberg, SM & Lee, S I 2017, 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, vol. 30, pp. 4765–4774, <https://arxiv.org/abs/1705.07874>

National Institute of Standards and Technology (NIST) 2023, Artificial intelligence risk management framework (AI RMF 1.0), NIST AI 100-1, https://doi.org/10.6028/NIST.AI.100_1

Open Worldwide Application Security Project (OWASP) 2023, OWASP API security top 10 – 2023, https://owasp.org/API_Security/editions/2023/en/0x11_t10/

Sculley, D, Holt, G, Golovin, D, Davydov, E, Phillips, T, Ebner, D, Chaudhary, V, Young, M, Crespo, J F & Dennison, D 2015, 'Hidden technical debt in machine learning systems', Advances in Neural Information Processing Systems, vol. 28, pp. 2503–2511, https://papers.nips.cc/paper/5656_hidden_technical_debt_in_machine_learning_systems